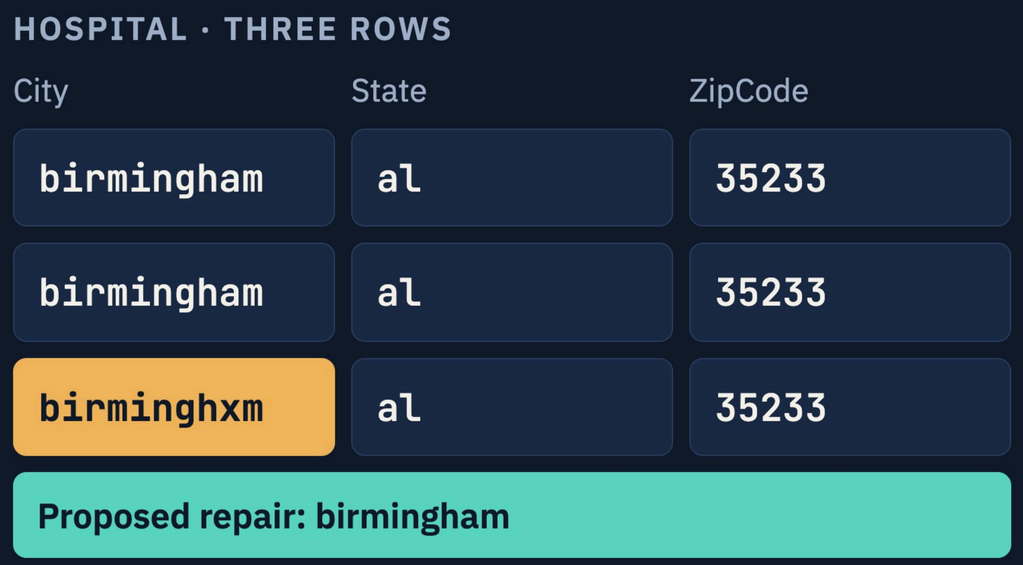
MILESTONE 1 · PROBLEM DEFINITION, LITERATURE REVIEW AND DATA AUDIT

ContextGuard

Finding and repairing cell-level errors in tables



Sidhaarth Shree · Jashan · Sahilpreet · Aaradhya

DSAI Lab Project, IIT Madras BS · Group 1

github.com/SidhaarthShree07/Group-1-DS-and-AI-Lab-Project

PROBLEM

The problem, and what is new

HOSPITAL EXAMPLE

A cell can look fine and still be wrong.

City	State	ZipCode
birminghxm	al	35233

48 rows share that zip; 45 say **birmingham**.

WHY IT MATTERS

Cleaning can help or hurt a downstream model (CleanML, 14 datasets), so we repair conservatively and measure the effect.

Approach today	Limit
Rules (NADEEF, HoloClean)	An expert writes constraints for every dataset; HoloClean repair F1 is ~0 on Beers and Rayyan (Baran paper)
Statistics	A rare value is often correct; a wrong value that looks ordinary is missed
Raha, Baran	Need labelled rows for every new table
LLM cleaners (ZeroED, Cocoon, ForestED)	Large hosted models, detection only, no calibrated confidence

ContextGuard — Cheap detectors choose the cells · Fine-tuned open 8B model decides: error? type? repair? · Calibrated gate applies only confident repairs

LANDSCAPE

What exists before LLMs

Rules and constraints

HoloClean · NADEEF · HyFD

They rely on functional dependencies and denial constraints. Rules are written by hand (9 constraints for Hospital); exact dependency discovery breaks on noisy tables.

0.01

HoloClean repair F1 on Beers; 0 on Rayyan

Statistical and learning-based cleaners

Raha · Baran · BClean

Ensembles of detectors plus up to 20 labelled tuples. They need fresh labels for every new table, and repair quality falls when detection is imperfect.

0.52 → 0.35

Rayyan F1: perfect detection vs end to end

Industry tools

Great Expectations · Deequ · Pandera · AWS Glue · Dataplex · Monte Carlo and others

Their documentation shows cross-column SQL checks, but every check is written by a person and detects only what is specified.

Rules a person writes

No documented automatic repair of cell values found

LANDSCAPE

What we must beat, and how we measure it

Numbers to beat · from the original papers, 20 labels · – = not quoted here

Method and metric	Hospital	Flights	Beers	Rayyan	Movies	Tax
Raha · detection F1 (Table 5)	0.72	0.81	0.99	0.79	0.86	–
Raha + Baran · repair F1 (Table 7)	0.66	0.66	0.90	0.35	–	0.74
dBoost alone · detection F1	0.49	0.66	0.55	–	–	–

HOW WE MEASURE

Error cell Differs from the clean value after Raha's normalisation.

Repair Counts only on an exact match.

Labels Raha and Baran get 20 labelled tuples per table; ContextGuard gets none on unseen tables.

Seeds Mean ± std over 10 seeds (42 to 51), reported on our own splits.

BASELINES

- **B1** HoloClean + TANE-mined rules
- **B2** dBoost
- **B3** Raha + Baran
- **B4** Gemma 4 E4B-it zero-shot
- **B5** Gemma 4 E4B-it with 3 examples

DATA AUDIT

Our benchmark tables and what we found in them

18,414

verified error cells in five tables

Raha rule: dirty ≠ clean after unescaping, whitespace collapse, strip.

Table	Size	Error cells	Rate
Hospital	1,000 × 19	509	2.68%
Flights	2,376 × 6	4,920	34.51%
Beers	2,410 × 10	4,362	18.10%
Rayyan	1,000 × 11	948	8.62%
Movies	7,390 × 17	7,675	6.11%

LOST ERRORS IN PUBLIC COPIES

The public TableEG copies had lost errors: **Beers 1,005** (N/A in the Raha dirty file, blank in TableEG's) and **Movies a net 668** (RatingValue 7.0 became 7). We use the Raha originals.

HEADERS DIFFER

Clean and dirty headers differ (Hospital 19, Movies 17, Beers 2), so columns are aligned by **position**, not by name.

DATA AUDIT

Error types, weak “clean” files and data sufficiency

81,596

true error cells across the nine audited tables



Missing 39,975 (49%) Pattern 37,810 (46%) Rule 2,329 (3%) Outlier 1,482 (2%)

Credit’s 315,466 float re-saving differences are kept apart.

Rule violations are scarce

Flights 1,149, Rayyan 619, Hospital 408, Beers 153. Training therefore adds synthetic dependency violations on clean UCI tables (at most 20% of training data).

“Clean” gold files still contain blanks

Beers 1,072, Movies 2,680 and Rayyan 1,609 cells, which limits achievable repair recall.

Unseen and downstream tables

Soccer: 796 errors typed by us (597 pattern, 199 outlier; BART’s log has no types). Restaurant: 633 errors, all pattern. Credit: imputed income 88.44070676 repeats 17,435 times; target 6.68% positive.

Enough data to fine-tune

The five core tables give an estimated 25,000–35,000 training examples, inside the 15,000–40,000 budget for LoRA fine-tuning of Gemma 4 E4B-it.

DATA AUDIT

The extra datasets that test generalisation

TABLE	ROLE	SIZE	ERROR CELLS	RATE
Tax (Raha)	Training	200,000 × 15	121,219	4.04%
Inpatient (BClean, 10%)	Training only	4,017 × 11	4,465	10.10%
Facilities (BClean, 10%)	Training only	7,992 × 10	8,318	10.41%
Billionaire (ZeroED)	Unseen test	2,614 × 22	5,195	9.03%
Clean Me If You Can, Germany	Unseen test	5,000 of 374,533 × 9	18,678	41.51%

Aligned and checked

Every pair was aligned and checked; BClean's dirty files have an extra row-index column that we drop.

Typed by us

The sources give no error types, so we type each changed cell: missing, format artifact, rule (a valid value in the wrong place) or pattern.

Held out

Unseen tables are never used for training, augmentation or profiling; Inpatient and Facilities share Hospital's domain, so they are training only.

DATA AUDIT

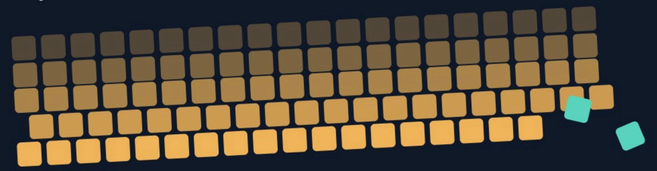
What we found, and the limits

TAX (TRAINING TABLE)

97.6%

of Tax's error cells are re-formatting

"3" against "3.0", and a leading zero on zip codes.



re-formatting real errors

Only **2,966 of 121,219** error cells are real injected errors, so these differences are kept out of the training positives.

Billionaire's clean file

200 "nan (wait to be cleaned)" placeholders.

Germany slice

Already published on Zenodo; 5,000 rows drawn with seed 42; 41.5% of cells are errors, mostly missing values.

No row overlap with training tables

No shared ids between Hospital and Inpatient or Facilities (one hospital name and address appears in both) and none between Billionaire and the other tables.

Licences

BClean and ZeroED state none, so they are cited and not redistributed; the address data is OpenStreetMap-derived (ODbL, share-alike, attribution); UCI tables for augmentation are CC BY 4.0.

OUR CONTRIBUTION

Objectives, evidence and the model choice

OBJECTIVES AND TARGETS

01 Detection
Best macro-F1 against all baselines

02 Safe repair
At most 5% of applied repairs wrong

03 Unseen tables
Above zero-shot LLM and Raha with 20 labels

04 Cost
Keeps most errors, sends only a minority to the LLM

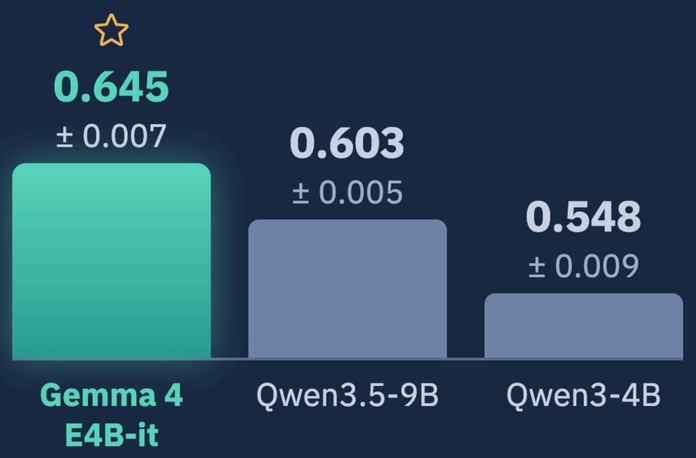
05 Downstream
Share of the dirty-versus-clean gap closed

06 End-to-end tool
Upload a CSV, get the cleaned file and a change log

BASE MODEL, ZERO-SHOT

bars start at 0.50

Gemma won every seed · 3 seeds of 1,500 cells, mean ± std



WRONG REPAIRS

lower is better

Gemma 4 E4B-it

0.39

Both Qwen models

0.68–0.70

DATA

18,414 verified error cells in the five core tables; 5 unseen test tables; Credit for the downstream test.

NEXT STEPS

Plan and risks

M2	**29 Oct**	**Dataset preparation.** Verified sources, splits with leakage checks, data card, first instruction–response examples
M3	**5 Nov**	**Model architecture.** Pipeline end to end on a subset, input formats, justification, loss and metrics
M4	**19 Nov**	**Model training.** Fine-tuning runs, ablations, several seeds
M5	**26 Nov**	**Evaluation and analysis.** Baseline comparison, precision–coverage curve, unseen tables, downstream experiment
M6	**10 Dec**	**Deployment and documentation.** Gradio demo and API, model and data cards, final report

Main risks

- The cheap stage misses many errors (recall ceiling).
- The model over-flags rare but correct values.
- Unclear licences (raw data stays out of the repository).
- Credit’s clean income values were imputed, so the downstream test is repeated on Beers.